ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN HỆ ĐIỀU HÀNH

BÁO CÁO THỰC HÀNH

MÔN HỆ ĐIỀU HÀNH

Lab 01

Xv6 User Programs: File System

Lớp học phần: Hệ điều hành - CQ2024/22

Sinh viên: 24120438 – Trần Nguyên Tân

24120023 – Nguyễn Thành Bảo

24120040 – Dương Hoài Đức

Giảng viên: ThS. Lê Viết Long

Mục lục

Tóm tắt báo cáo **3**

1 Thông tin thành viên và phân công công việc **4**

2 Thiết lập môi trường trên Windows và macOS **4**

2.1 Windows: Ubuntu và dependencies trên WSL 2 4

2.2 macOS: cài đặt công cụ RISC-V qua Homebrew 5

2.3 Chuẩn bị mã nguồn và khởi chạy xv6 6

2.4 Môi trường dùng để ghi nhận kết quả 6

3 Kiến thức nền tảng **7**

3.1 Mô hình thực thi chương trình người dùng 7

3.2 Các system call liên quan 9

3.3 Các hàm thư viện người dùng 12

3.4 Tiến trình và cơ chế giao tiếp liên tiến trình qua đường ống 15

3.5 Hệ thống tập tin và kỹ thuật duyệt cây thư mục 16

3.6 Xử lý dòng dữ liệu I/O và nguyên tắc quản trị tài nguyên an toàn . 19

4 Các chương trình đã thực hiện **20**

4.1 sleep 20

4.2 pingpong 21

4.3 primes 23

4.4 cp 25

4.5 tree 26

4.6 du 28

4.7 diff 29

5 Kết quả kiểm thử tổng hợp **31**

5.1 Môi trường và cách thực hiện 31

5.2 Kết quả theo từng chương trình 32
5.3 Nhật ký thực thi và phạm vi kết luận 33

6 Khó khăn gặp phải và giải pháp khắc phục 33

7 Kết luận và hướng phát triển 34

Tài liệu tham khảo 34

Tóm tắt báo cáo

Báo cáo này trình bày kết quả thực hiện, thiết kế kỹ thuật và quá trình kiểm thử toàn diện các chương trình tiện ích không gian người dùng (*User Programs*) trong khuôn khổ bài thực hành **Lab 01 (Hệ điều hành xv6 – Kiến trúc RISC-V)**. Mục tiêu trọng tâm của bài thực hành là tiếp cận kiến trúc phân tầng người dùng/kernel (*User/Kernel Space*), vận dụng các system call cơ bản, cơ chế quản lý vòng đời tiến trình, truyền thông liên tiến trình (IPC) và các thao tác truy xuất hệ thống tệp tin (*File System*).

Trong suốt quá trình thực hiện, nhóm đã phân tích yêu cầu, xây dựng mã giả và hiện thực thành công toàn bộ **07 chương trình tiện ích**:

- sleep: Tạm dừng tiến trình theo chu kỳ đồng hồ (*ticks*) của hệ thống.
- pingpong: Hiện thực cơ chế giao tiếp 2 chiều đồng bộ giữa tiến trình cha và con thông qua cặp đường ống (*pipes*).
- primes: Xây dựng thuật toán sàng số nguyên tố Eratosthenes đồng thời (*Concurrent Prime Sieve*) theo mô hình pipeline đa tiến trình kết nối liên tiếp từ 2 đến 280.
- cp: Sao chép dữ liệu tập tin an toàn qua bộ đệm (*buffer streaming*), xử lý kiểm tra trùng tệp nguồn/đích.
- tree: Duyệt đệ quy cây thư mục hiển thị phân cấp trực quan, hỗ trợ giới hạn độ sâu (-L) và lọc thư mục (-d).
- du: Ước tính dung lượng sử dụng đĩa (*Disk Usage*) theo byte, hỗ trợ tính tổng nhanh (-s) và liệt kê chi tiết tệp (-a).
- diff: So sánh nội dung hai tệp tin theo từng dòng, xử lý so khớp EOF và chế độ thông báo tóm tắt (-q).

Tóm tắt kết quả & Đánh giá chung:

- **Tỷ lệ hoàn thành:** 7/7 chương trình (100% khối lượng yêu cầu).
- **Độ tin cậy & Quản lý tài nguyên:** Tất cả các chương trình đều được kiểm thử nghiêm ngặt trên trình giả lập QEMU, kiểm soát chặt chẽ việc giải phóng bộ mô tả tập tin (không rò rỉ File Descriptor), đóng đúng các đầu pipe thừa để kích hoạt tín hiệu EOF chính xác, và đồng bộ hóa triệt để bằng wait() nhằm loại bỏ hoàn toàn nguy cơ sinh tiến trình ma (*zombie process*).
- **Kiểm thử tự động:** Đáp ứng đầy đủ các trường hợp kiểm thử chức

> năng và edge cases lỗi theo chuẩn kịch bản kiểm thử của môn học.

1. Thông tin thành viên và phân công công việc

STT	MSSV	Họ và tên	Nhiệm vụ phụ trách
1	24120438	Trần Nguyên Tân	sleep, pingpong, primes
2	24120023	Nguyễn Thành Bảo	cp, diff
3	24120040	Dương Hoài Đức	tree, du, tích hợp và kiểm thử chung

2. Thiết lập môi trường trên Windows và macOS

Do xv6 trong bài thực hành chạy trên kiến trúc RISC-V. Máy chủ đảm nhiệm việc biên dịch mã nguồn, còn QEMU mô phỏng phần cứng để khởi động xv6. Vì vậy, dù dùng Windows hay macOS, môi trường cần có trình biên dịch chéo RISC-V, bộ công cụ liên kết và QEMU; trình biên dịch C mặc định của máy không thay thế được bộ công cụ RISC-V.

2.1 Windows: Ubuntu và dependencies trên WSL 2

Trên Windows, quá trình biên dịch và kiểm thử được thực hiện trong Ubuntu trên WSL 2. Cách tổ chức này giữ được môi trường Linux theo yêu cầu bài thực hành mà vẫn cho phép soạn thảo tài liệu và quản lý mã nguồn từ Windows. Từ hướng dẫn của MIT, nhóm xác định sử dụng WSL 2 và QEMU từ phiên bản 7.2; Ubuntu phiên bản 24.04 để thuận tiện cài các gói phù hợp [6].

Cài đặt WSL. Mở PowerShell với quyền Administrator, cài Ubuntu rồi khởi động lại máy nếu được yêu cầu. Khi mở Ubuntu lần đầu, tạo tài khoản và mật khẩu Linux. Các lệnh dưới đây áp dụng cho Windows 10 phiên bản 2004 trở lên hoặc Windows 11 [9].

```
wsl --install -d Ubuntu-24.04
wsl --list --verbose
```

Cài công cụ phát triển. GCC và Binutils cho RISC-V dùng để tạo chương trình chạy trên xv6; build-essential cung cấp công cụ biên dịch phía máy chủ, QEMU

chạy máy mô phỏng, còn Python dùng cho bộ kiểm thử của dự án. gdb-multiarch hỗ trợ gỡ lỗi khi cần.

```
sudo apt update
sudo apt install build-essential git perl python3 \
  gdb-multiarch qemu-system-misc \
  gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu

riscv64-linux-gnu-gcc --version
qemu-system-riscv64 --version
python3 --version
```

2.2 macOS: cài đặt công cụ RISC-V qua Homebrew

Trên macOS, có thể biên dịch trực tiếp trong Terminal và chạy xv6 bằng QEMU. Trước hết, cài Command Line Tools để có công cụ phát triển phía máy chủ, sau đó cài Homebrew theo hướng dẫn chính thức [10].

```
xcode-select --install
```

Sau khi cài Homebrew, thực hiện phần cấu hình brew shellenv được trình cài đặt hướng dẫn rồi mở lại Terminal. Đường dẫn này phụ thuộc vào cách cài và kiến trúc máy; không nên sao chép một đường dẫn cố định từ máy khác.

```
brew --version
brew install riscv64-elf-gcc qemu python

riscv64-elf-gcc --version
riscv64-elf-objdump --version
qemu-system-riscv64 --version
python3 --version
```

Gói riscv64-elf-gcc của Homebrew kéo theo Binutils tương ứng [11]. Đây là cách cài được dùng trong hướng dẫn này, thay cho cấu hình tap RISC-V trong tài liệu cũ. Khả năng cung cấp gói dựng sẵn phụ thuộc phiên bản macOS và kiến trúc máy, nên cần kiểm tra điều kiện hỗ trợ của Homebrew trước khi cài.

Bộ công cụ này có tiền tố riscv64-elf-, trong khi Makefile của mã nguồn hiện tại chưa tự nhận diện tiền tố đó. Vì vậy, khi biên dịch trên macOS phải truyền rõ TOOLPREFIX, thay vì đổi tên các chương trình hoặc sửa cố định đường dẫn trong Makefile.

2.3 Chuẩn bị mã nguồn và khởi chạy xv6

Nhóm đã sử dụng mã nguồn trong folder xv6-labs-2024/ của dự án, source code của xv6 được clone từ git://g.csail.mit.edu/xv6-labs-2024

Từ thư mục gốc dự án, chạy các lệnh sau để biên dịch:

Ubuntu trên WSL 2

```
cd xv6-labs-2024
make
make qemu
```

macOS

```
cd xv6-labs-2024
make TOOLPREFIX=riscv64-elf-
make TOOLPREFIX=riscv64-elf- qemu
```

Môi trường sẵn sàng khi xv6 khởi động sẽ xuất hiện init: starting sh và dấu nhắc shell. Khi đó ta có thể chạy các lệnh sleep 10, pingpong hoặc primes để kiểm tra bước đầu. Để thoát QEMU, nhấn Ctrl+A, thả phím rồi nhấn X.

Các chương trình người dùng được khai báo trong UPROGS của Makefile. Ví dụ, tệp dựng user/_sleep được đưa vào fs.img với tên sleep; khi chạy trong shell xv6 không thêm dấu gạch dưới. Các tệp tạo trong xv6 thuộc ảnh đĩa này, không xuất hiện như tệp thông thường trong thư mục máy chủ. Lệnh make clean xóa sản phẩm biên dịch và ảnh đĩa, nên cần lưu dữ liệu thử nghiệm quan trọng trước khi dọn bản dựng.

Kiểm thử tự động. Từ thư mục gốc dự án, dùng python3 tests/lab01/run.py trên WSL. Trên macOS, thêm tùy chọn chọn bộ công cụ:

```
python3 tests/lab01/run.py --toolprefix=riscv64-elf-
```

2.4 Môi trường dùng để ghi nhận kết quả

Các thao tác kiểm thử được báo cáo ở mục 5 được thực hiện trong Ubuntu 24.04 qua WSL, với môi trường cài đặt GCC RISC-V 13.3.0 và QEMU 8.2.2. Đây là môi trường đã tạo ra kết quả 77/77 ca kiểm thử của báo cáo.

3. Kiến thức nền tảng

Để xây dựng các chương trình tiện ích không gian người dùng (*user programs*) trong bài thực hành Lab 01, nhóm đã tìm hiểu và vận dụng các kiến thức cốt lõi về giao diện hệ điều hành và hệ thống tập tin được trình bày trong tài liệu *xv6 book* (đặc biệt là Chương 1: *Operating system interfaces* và Chương 8: *File system*) [1].

Mục này trình bày có hệ thống các cơ chế nền tảng của xv6 liên quan trực tiếp đến bài thực hành, bao gồm: mô hình phân tầng không gian người dùng và kernel, hệ thống lời gọi hệ thống (system call) cùng thư viện người dùng, cơ chế quản lý vòng đời tiến trình và giao tiếp qua đường ống (pipe), cấu trúc tổ chức của hệ thống tập tin (inode, directory entry), cũng như các kỹ thuật xử lý I/O và nguyên tắc quản trị tài nguyên an toàn.

3.1 Mô hình thực thi chương trình người dùng

Hệ điều hành xv6 vận hành trên kiến trúc vi xử lý RISC-V, tận dụng cơ chế phân quyền phần cứng để chia tách thành hai chế độ thực thi: *chế độ người dùng* (User mode) và *chế độ giám sát* (Supervisor mode). Tất cả các tiện ích người dùng (`sleep`, `pingpong`, `cp`,...) đều được nạp và thực thi trong **không gian người dùng** (*user space*). Tại đây, mỗi tiến trình sở hữu một không gian địa chỉ ảo riêng biệt được ánh xạ thông qua bảng trang (*page table*). Cơ chế phân lập này giúp bảo vệ tính toàn vẹn của hệ thống: một chương trình người dùng không thể can thiệp hay đọc trộm vùng nhớ của tiến trình khác, đồng thời không thể truy cập trực tiếp vào các cấu trúc dữ liệu nội bộ của kernel.

Khi cần thao tác với phần cứng hoặc tài nguyên dùng chung—chẳng hạn như đọc ghi dữ liệu trên đĩa, tạm dừng theo nhịp đồng hồ, hay truyền dữ liệu cho tiến trình khác—chương trình phải phát ra một **lời gọi hệ thống** (*system call*). Trên nền tảng RISC-V, thao tác này được kích hoạt thông qua chỉ thị `ecall`, tạo ra một bẫy phần mềm (*trap*) chuyển quyền kiểm soát sang **không gian kernel** (*kernel space*). Kernel sẽ xác thực tham số, thực thi tác vụ tương ứng với đầy đủ đặc quyền phần cứng rồi trả kết quả về cho tiến trình người dùng tiếp tục luồng thực thi.

Cơ chế nạp lệnh của Shell: Sự phối hợp giữa fork() và exec()

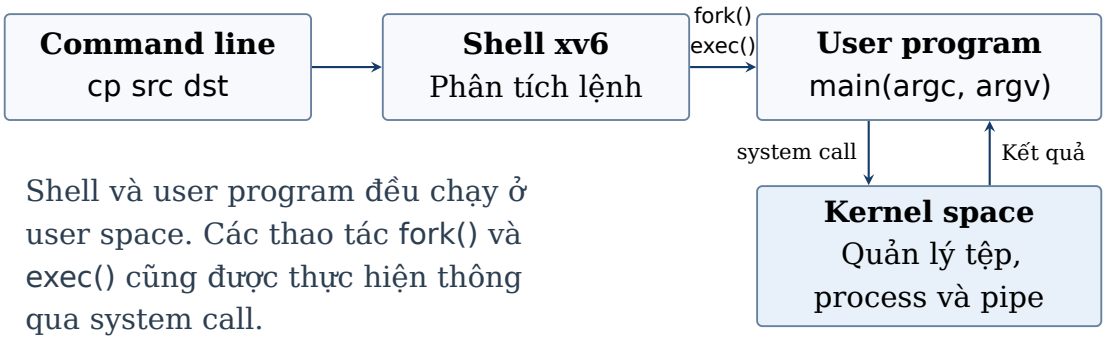
Điểm độc đáo trong thiết kế của Unix được xv6 kế thừa trọn vẹn là sự tách biệt rõ ràng giữa hai tác vụ: tạo tiến trình mới và nạp mã chương trình thực thi. Khi người dùng nhập một câu lệnh vào dấu nhắc shell (chẳng hạn `cp src dst`):

1. Shell (vốn cũng là một tiến trình chạy ở user space) đọc và phân tích cú pháp chuỗi dòng lệnh thành tên chương trình (`"cp"`) cùng mảng các đối số

(argv).

2. Shell gọi system call fork() để tạo một tiến trình con (*child process*). Tiến trình con này ban đầu là bản sao chính xác về không gian nhớ và trạng thái các tập tin mô tả đang mở của tiến trình cha.
3. Trong tiến trình con, shell gọi tiếp system call exec() để thay thế toàn bộ ảnh bộ nhớ hiện tại bằng mã máy nhị phân của chương trình cp được nạp từ hệ thống tập tin, sau đó bắt đầu thực thi từ hàm main().
4. Trong khi tiến trình con thực hiện nhiệm vụ sao chép, tiến trình shell cha gọi wait() để tạm dừng cho đến khi tiến trình con hoàn tất và trả về mã trạng thái thoát, rồi mới hiển thị lại dấu nhắc lệnh.

Sự tách biệt giữa fork() và exec() mang lại tính linh hoạt rất lớn: nó cho phép shell can thiệp chuyển hướng I/O (*I/O redirection*) hoặc thiết lập đường ống (*pipe*) ngay trong tiến trình con *trước khi* mã chương trình mới thực sự được nạp, mà không làm xáo trộn cấu hình của tiến trình cha hay đòi hỏi kernel phải can thiệp vào logic của từng lệnh.



Luồng khởi chạy một lệnh và tương tác giữa user program với kernel.

Xử lý tham số dòng lệnh (argc, argv)

Khi nạp một chương trình qua exec(), kernel thiết lập sẵn trên đỉnh ngăn xếp (stack) của tiến trình mới hai tham số: số lượng đối số argc và mảng con trỏ chuỗi ký tự argv. Theo chuẩn ngôn ngữ C trong xv6:

- argv[0] luôn là chuỗi chứa tên lệnh được gọi (chẳng hạn "sleep", "cp").
- Các phần tử tiếp theo từ argv[1] đến argv[argc-1] chứa các đối số dòng lệnh mà người dùng truyền vào.
- Phần tử kết thúc mảng argv[argc] luôn là một con trỏ NULL.

Vì toàn bộ đối số ban đầu đều ở dạng chuỗi ký tự thô, các chương trình cần tham số định lượng (như số nhịp chờ của sleep) hoặc các cờ tùy chọn (như -L trong tree, -s trong du, -q trong diff) bắt buộc phải tự kiểm tra tính hợp lệ về

cú pháp, phòng ngừa trường hợp thiếu đối số, truyền ký tự không hợp lệ hoặc vượt ngưỡng cho phép trước khi thực thi nghiệp vụ chính.

3.2 Các system call liên quan

System call là giao diện lập trình duy nhất kết nối giữa ứng dụng người dùng và kernel. Trong xv6, mã nguồn kernel định nghĩa danh sách system call trong kernel/syscall.h và cài đặt hàm xử lý tại kernel/sysproc.c và kernel/sysfile.c.

Tập tin mô tả (File Descriptors) và tính trừu tượng I/O

Theo triết lý Unix, xv6 trừu tượng hóa hầu hết các đối tượng I/O—từ tệp tin thông thường, thư mục, thiết bị hiển thị console cho đến đường ống liên tiến trình—dưới dạng một chuỗi byte truy xuất thông qua một số nguyên nhỏ gọi là **tập tin mô tả** (*file descriptor - fd*).

Mỗi tiến trình duy trì một bảng các tập tin đang mở (proc->ofile[NOFILE]). Giá trị fd mà tiến trình nhận được thực chất là một chỉ mục (*index*) trỏ vào bảng này. Theo quy ước mặc định:

- fd = 0: Luồng nhập chuẩn (*Standard Input - stdin*), nhận dữ liệu từ bàn phím.
- fd = 1: Luồng xuất chuẩn (*Standard Output - stdout*), in dữ liệu ra màn hình.
- fd = 2: Luồng báo lỗi chuẩn (*Standard Error - stderr*), in thông báo lỗi.

Một nguyên tắc cốt lõi trong xv6 là kernel luôn cấp phát **chỉ số descriptor nhỏ nhất chưa được sử dụng** (*lowest-numbered unused descriptor*). Đặc tính này cho phép shell dễ dàng thực hiện chuyển hướng I/O: chỉ cần gọi close(0) rồi gọi open("input.txt", O_RDONLY), tệp vừa mở sẽ tự động chiếm lấy fd = 0.

Dưới đây là bảng tổng hợp các system call được sử dụng trực tiếp trong bài thực hành, phân chia theo nhóm chức năng:

Nhóm quản lý tiến trình, thời gian chờ và IPC:

System call	Chức năng và tham số	Kết quả trả về và lưu ý kỹ thuật
fork()	Tạo một tiến trình con mới nhân bản từ tiến trình hiện tại.	Trả về 0 trong tiến trình con; trả về PID của con trong tiến trình cha; trả về -1 nếu thất bại. Cả hai tiến trình cùng tiếp tục chạy độc lập từ sau vị trí gọi fork().
wait(status)	Tạm dừng tiến trình cha cho đến khi một tiến trình con kết thúc; thu hồi tài nguyên và mã thoát.	Trả về PID của tiến trình con vừa kết thúc, hoặc -1 nếu không còn con để chờ. Nếu con trỏ status khác NULL, kernel sẽ ghi mã thoát của con vào địa chỉ này. Dùng wait(0) khi không cần nhận mã thoát.
exit(status)	Chấm dứt tiến trình gọi với mã trạng thái status.	Không bao giờ quay lại nơi gọi. Kernel tự động đóng các file descriptor còn mở của tiến trình và chuyển sang trạng thái zombie để chờ cha thu hồi.
getpid()	Lấy mã định danh (PID) của tiến trình đang thực thi.	Trả về số nguyên dương PID; dùng để nhận diện và phân biệt giữa tiến trình cha và con trong kết quả hiển thị (như trong pingpong).
sleep(ticks)	Tạm dừng tiến trình trong một số lượng chu kỳ ngắt định thời.	Trả về 0 khi hoàn tất thời gian chờ, -1 nếu bị hủy. Đơn vị ticks tính theo xung nhịp ngắt của phần cứng, không tương đương với giây thời gian thực.
pipe(p)	Tạo kênh truyền dữ liệu một chiều; ghi hai descriptor vào mảng p.	Trả về 0 khi thành công, -1 nếu xảy ra lỗi. p[0] là đầu đọc (*read end*), p[1] là đầu ghi (*write end*).

Nhóm thao tác tập tin, I/O và nạp chương trình:

System call	Chức năng và tham số	Kết quả trả về và lưu ý kỹ thuật
open(path, mode)	Mở hoặc tạo một tập tin theo đường dẫn với chế độ truy cập xác định.	Trả về file descriptor không âm nhỏ nhất còn trống nếu thành công; trả về -1 nếu thất bại. Các cờ chế độ (O_RDONLY, O_CREATE,...) được định nghĩa trong kernel/fcntl.h.
close(fd)	Đóng tập tin mô tả fd và giải phóng vị trí tương ứng trong bảng quản lý của tiến trình.	Trả về 0 khi thành công, -1 nếu descriptor không hợp lệ. Đối với pipe, đóng đầu ghi là điều kiện tiên quyết để bên đọc nhận biết tín hiệu kết thúc dữ liệu (EOF).
read(fd, buf, n)	Đọc tối đa n byte từ tệp, thiết bị hoặc pipe vào vùng đệm buf.	Trả về số byte thực tế đọc được; trả về 0 khi chạm điểm kết thúc dữ liệu (EOF); trả về -1 khi gặp lỗi I/O. Dữ liệu đọc vào không tự động kèm ký tự kết thúc chuỗi NULL.
write(fd, buf, n)	Ghi n byte từ vùng đệm buf ra tập tin mô tả fd.	Trả về số byte thực tế đã ghi, hoặc -1 nếu xảy ra lỗi. Cần kiểm tra số byte thực tế để đảm bảo toàn bộ nội dung đã được ghi đầy đủ vào thiết bị lưu trữ.
fstat(fd, st)	Lấy thông tin siêu dữ liệu của tập tin đang mở vào cấu trúc struct stat.	Trả về 0 khi thành công, -1 khi thất bại. Kết quả trả về gồm loại tệp (type), số hiệu thiết bị (dev), số Inode (ino) và kích thước tệp theo byte (size).
exec(path, argv)	Thay thế tiến trình hiện tại bằng chương trình thực thi mới tại đường dẫn path.	Giữ nguyên PID và các file descriptor đang mở. Nếu thành công, lời gọi không bao giờ quay lại nơi gọi; trả về -1 nếu không tìm thấy tệp hoặc lỗi định dạng nhị phân.

3.3 Các hàm thư viện người dùng

Bên cạnh các system call, xv6 cung cấp một thư viện người dùng tối giản được khai báo trong user/user.h (triển khai tại user/ulib.c, user/printf.c, user/umalloc.c). Cần phân biệt rõ hai khái niệm này: system call yêu cầu sự can thiệp của kernel và tốn chi phí chuyển ngữ cảnh (trap context switch), trong khi các hàm thư viện người dùng chạy hoàn toàn trong user space và được liên kết tĩnh trực tiếp vào mã nhị phân của chương trình.

Một số hàm thư viện đóng vai trò là hàm bọc tiện ích (*wrapper*). Điển hình như stat(path, st): trong kernel xv6 không có system call nhận trực tiếp chuỗi đường dẫn để lấy thông tin tệp, mà chỉ có fstat(fd, st) thao tác trên file descriptor. Hàm thư viện stat() được xây dựng bằng cách gọi liên tiếp open(), fstat() và tự động gọi close() trước khi trả về.

Do thư viện của xv6 không đầy đủ như chuẩn GNU libc hay POSIX, các tiện ích phức tạp trong bài (chẳng hạn như đọc từng dòng trong diff hay duyệt phân cấp trong tree) đòi hỏi người lập trình phải tự xây dựng giải pháp xử lý dữ liệu và quản lý bộ nhớ dựa trên các hàm cơ bản sau:

Hàm thư viện	Công dụng	Điểm cần lưu ý khi sử dụng
Thao tác đường dẫn, chuỗi ký tự và dữ liệu		
stat(path, st)	Lấy thông tin siêu dữ liệu của tệp theo chuỗi đường dẫn.	Là hàm bọc gọi liên tiếp open(), fstat() và close(). Trả về 0 khi thành công, -1 khi lỗi. Tự động đóng descriptor trước khi thoát.
atoi(s)	Chuyển đổi chuỗi ký tự chữ số thành số nguyên kiểu int.	Không cung cấp cơ chế báo lỗi khi chuỗi chứa ký tự phi số; chương trình cần chủ động kiểm tra tính hợp lệ của chuỗi đầu vào trước khi gọi hàm.
strcmp(a, b)	So sánh hai chuỗi ký tự kết thúc bằng ký tự NULL.	Trả về 0 khi hai chuỗi giống hệt nhau; dùng để nhận diện cờ lệnh (-L, -d, -s,...) và phát hiện các thư mục đặc biệt (".", "...").
strlen(s)	Tính độ dài chuỗi ký tự (không bao gồm ký tự NULL).	Chỉ áp dụng an toàn cho các chuỗi có ký tự kết thúc chuẩn; không dùng để suy ra kích thước của khối dữ liệu nhị phân hoặc vùng đệm chưa gán '\0'.
memcmp(a, b, n)	So sánh chính xác n byte giữa hai vùng nhớ.	Trả về 0 nếu hai vùng nhớ giống nhau; đặc biệt phù hợp để so sánh các dòng văn bản có chứa byte NULL hoặc dữ liệu nhị phân trong diff.

Hàm thư viện	Công dụng	Điểm cần lưu ý khi sử dụng
Cấp phát bộ nhớ và xuất dữ liệu định dạng		
memmove(dst, src, n)	Sao chép an toàn n byte từ vùng nhớ nguồn sang vùng nhớ đích.	Hỗ trợ trường hợp hai vùng nhớ bị chồng lấn (*overlapping*); cần đảm bảo vùng nhớ đích đã được cấp phát đủ dung lượng tối thiểu n byte.
malloc(n), free(p)	Cấp phát và giải phóng bộ nhớ động trên vùng nhớ heap.	Bắt buộc kiểm tra con trỏ trả về từ malloc() để phòng ngừa lỗi cạn bộ nhớ; luôn giải phóng bộ nhớ bằng free() khi mở rộng hoặc tái sử dụng vùng đệm.
printf(fmt, ...), fprintf(fd, fmt, ...)	In chuỗi định dạng ra luồng xuất chuẩn (fd = 1) hoặc descriptor chỉ định.	Hỗ trợ các định dạng cơ bản: %d, %x, %p, %s, %c. Trong xv6, hai hàm này trả về kiểu void thay vì số ký tự như trong libc chuẩn.

3.4 Tiến trình và cơ chế giao tiếp liên tiến trình qua đường ống

Để xây dựng các ứng dụng đa tiến trình như pingpong và primes, việc hiểu rõ cách thức hệ điều hành tạo lập, điều phối và truyền thông giữa các tiến trình là yếu tố quyết định.

Quản lý tiến trình và đồng bộ hóa qua wait()

Trong xv6, mỗi tiến trình là một thực thể độc lập được đại diện bởi cấu trúc struct proc trong kernel, bao gồm trạng thái thực thi (RUNNING, RUNNABLE, SLEEPING, ZOMBIE), định danh tiến trình (PID), không gian địa chỉ bảng trang, và con trỏ liên kết đến tiến trình cha. Lời gọi fork() sao chép toàn bộ không gian địa chỉ của cha sang một tiến trình con mới, đồng thời sao chép toàn bộ bảng các tập tin mô tả đang mở. Sau fork(), cả hai tiến trình tiếp tục chạy từ vị trí lệnh kế tiếp nhưng trong hai không gian nhớ riêng biệt.

Khi tiến trình con hoàn tất công việc, nó gọi exit(status). Lúc này, kernel thu hồi hầu hết tài nguyên bộ nhớ của con nhưng vẫn giữ lại cấu trúc proc của nó ở trạng thái ***ZOMBIE*** để lưu lại mã thoát. Tiến trình cha bắt buộc phải gọi wait() để thu nhận mã trạng thái này và cho phép kernel giải phóng hoàn toàn tiến trình con khỏi bảng tiến trình. Nếu tiến trình cha không gọi wait(), tiến trình con sẽ trở thành tiến trình ma (*zombie process*), gây rò rỉ ô nhớ trong bảng tiến trình hệ thống.

Cơ chế hoạt động của Pipe và quy tắc EOF sống còn

Đường ống (*pipe*) là kênh truyền dữ liệu một chiều giữa các tiến trình trong Unix. Trong nhân xv6, một pipe được hiện thực bằng một bộ đệm tròn (*circular buffer*) có kích thước cố định (512 byte). System call pipe(p) tạo ra đường ống và trả về hai tập tin mô tả: p[0] dùng để đọc dữ liệu và p[1] dùng để ghi dữ liệu. Dữ liệu ghi vào p[1] sẽ được lưu tạm trong bộ đệm của kernel cho đến khi tiến trình đọc lấy ra từ p[0] theo cơ chế FIFO.

Hành vi đồng bộ hóa của pipe tuân thủ các quy tắc nghiêm ngặt:

- Khi bộ đệm rỗng, thao tác read() trên đầu đọc sẽ bị tạm dừng (*block*), đưa tiến trình vào trạng thái ngủ để chờ dữ liệu được ghi vào.
- Khi bộ đệm đầy, thao tác write() trên đầu ghi cũng sẽ bị tạm dừng cho đến khi tiến trình đọc giải phóng bớt dung lượng trong bộ đệm.
- **Quy tắc nhận diện EOF (End-of-File):** Thao tác read() chỉ trả về 0 (tín hiệu kết thúc dữ liệu) khi và chỉ khi bộ đệm của pipe đã rỗng **VÀ toàn bộ các tập tin mô tả trỏ đến đầu ghi p[1] trên toàn hệ thống đã được đóng**.

Quy tắc EOF trên là điểm mấu chốt quyết định sự thành bại khi lập trình với pipe. Do tiến trình con sau fork() kế thừa toàn bộ các file descriptor của cha, nên cả hai tiến trình đều đồng thời giữ các bản sao của p[0] và p[1]. Nếu một tiến trình chỉ có nhiệm vụ đọc dữ liệu nhưng quên không đóng đầu ghi p[1] của chính nó, hệ thống vẫn ghi nhận còn ít nhất một đầu ghi đang mở. Do đó, khi bên kia ngừng gửi, thao tác read() sẽ không bao giờ nhận được giá trị 0 mà bị treo vĩnh viễn trong trạng thái chờ (*deadlock*).

Trong bài pingpong, việc đóng ngay các đầu pipe không sử dụng giúp hai tiến trình chuyển giao quyền thực thi tuần tự và tin cậy. Trong bài primes—hiện thực thuật toán sàng số nguyên tố đồng thời nổi tiếng của Doug McIlroy—các tiến trình con được kết nối nối tiếp nhau thành một chuỗi pipeline dài. Mỗi tiến trình đọc số nguyên tố đầu tiên từ đường ống bên trái, in ra màn hình, rồi tạo một tiến trình con bên phải để chuyển tiếp các số không chia hết cho số nguyên tố đó. Việc đóng triệt để đầu ghi cũ trước khi đọc dữ liệu mới là điều kiện bắt buộc duy nhất để tín hiệu EOF có thể lan truyền xuyên suốt toàn bộ chuỗi pipeline, giúp các tiến trình tự động kết thúc tuần tự từ trái sang phải mà không làm cạn kiệt bảng file descriptor của hệ thống.

Cơ chế nhịp đồng hồ và ngắt định thời của sleep()

Khác với hàm sleep() trong các hệ điều hành thông thường vốn nhận tham số tính theo giây, system call sleep(ticks) trong xv6 hoạt động dựa trên nhịp ngắt của bộ định thời phần cứng (*timer interrupt*). Sau mỗi khoảng thời gian định kỳ, vi xử lý RISC-V kích hoạt một ngắt hẹn giờ, chuyển quyền điều khiển cho hàm xử lý ngắt trong kernel để tăng biến toàn cục ticks lên 1 đơn vị, đồng thời đánh thức các tiến trình đang chờ trên biến này.

Khi chương trình gọi sleep(n), kernel chuyển tiến trình sang trạng thái ngủ và nhường CPU cho các tiến trình khác. Sau khi đủ n nhịp ngắt trôi qua, tiến trình mới được chuyển lại sang trạng thái sẵn sàng chạy (RUNNABLE) để bộ lập lịch cấp phát CPU. Cơ chế này loại bỏ hoàn toàn việc lãng phí chu kỳ xử lý do vòng lặp bận (*busy-waiting*), nhưng cũng đồng nghĩa với việc khoảng thời gian chờ thực tế phụ thuộc vào tần số ngắt của máy ảo QEMU và tải của hệ thống, không phản ánh trực tiếp thời gian thực tính bằng giây.

3.5 Hệ thống tập tin và kỹ thuật duyệt cây thư mục

Hệ thống tập tin của xv6 tổ chức dữ liệu theo mô hình phân cấp dạng cây kinh điển của Unix. Để thao tác chính xác với tập tin và thư mục trong các tiện ích cp, tree, du, và diff, việc nắm vững cấu trúc nhị phân của Inode và Directory Entry là nền tảng tối quan trọng.

Phân biệt Inode và Tên đường dẫn (Pathname)

Trong hệ thống tập tin xv6, mỗi tập tin được đại diện trên đĩa bởi một cấu trúc dữ liệu gọi là ***Inode*** (*index node*). Inode lưu trữ toàn bộ thông tin siêu dữ liệu (*metadata*) của tệp bao gồm: loại tệp (type: tệp thường T_FILE, thư mục T_DIR, hay thiết bị T_DEVICE), số hiệu thiết bị lưu trữ (dev), số Inode (ino), số lượng liên kết cứng (nlink), kích thước tệp tính theo byte (size), và địa chỉ các khối dữ liệu trên đĩa.

Điểm cốt lõi là: **Inode không lưu tên của tệp tin**. Tên tệp chỉ tồn tại bên trong các thư mục dưới dạng các mục tham chiếu. Do cơ chế liên kết cứng (*hard link*), nhiều tên đường dẫn khác nhau ở các vị trí khác nhau trong cây thư mục hoàn toàn có thể cùng trỏ về một Inode duy nhất trên đĩa.

Kiến thức này trực tiếp giải quyết bài toán an toàn dữ liệu trong tiện ích cp: Khi sao chép từ tệp nguồn sang tệp đích, nếu người dùng vô tình chỉ định hai đường dẫn trỏ về cùng một tệp vật lý (chẳng hạn cp a.txt a.txt hoặc qua hai liên kết cứng khác nhau), việc mở tệp đích với cờ cắt cụt (O_TRUNC) sẽ lập tức xóa sạch nội dung của tệp nguồn trước khi kịp sao chép, dẫn đến mất dữ liệu không thể phục hồi. Do đó, thay vì so sánh hai chuỗi đường dẫn (vốn dễ bị đánh lừa bởi đường dẫn tương đối), chương trình phải dùng stat() để đọc thông tin của cả hai tệp và kiểm tra cặp định danh vật lý duy nhất:

$$\text{st_src.dev} == \text{st_dst.dev} \quad \text{và} \quad \text{st_src.ino} == \text{st_dst.ino}$$

Cấu trúc thư mục và bản ghi struct dirent

Trong kiến trúc của xv6, một thư mục thực chất là một tệp tin đặc biệt mà nội dung bên trong là danh sách liên tiếp các bản ghi cấu trúc thư mục (struct dirent), được định nghĩa trong kernel/fs.h:

- ushort inum: Số hiệu Inode mà mục này trỏ tới (2 byte).
- char name[DIRSIZ]: Tên của tệp hoặc thư mục con, có độ dài cố định tối đa 14 ký tự (DIRSIZ = 14).

Khi một chương trình mở thư mục bằng open(path, O_RDONLY), nó có thể dùng chính system call chuẩn read() để đọc tuần tự từng khối dữ liệu có kích thước đúng bằng sizeof(struct dirent). Khi phân tích danh sách này:

- Nếu bản ghi có de.inum == 0, điều đó có nghĩa là mục này đang trống hoặc đã bị xóa, chương trình cần bỏ qua.
- Chuỗi de.name có độ dài tối đa 14 ký tự và không được đảm bảo luôn có ký tự kết thúc chuỗi '\0' nếu tên tệp chiếm đủ 14 ký tự. Vì vậy, trước khi gọi các hàm xử lý chuỗi chuẩn như strcmp() hay in ra màn hình, chương trình

bắt buộc phải sao chép tên sang một bộ đệm an toàn và bảo đảm chèn ký tự '\0' vào cuối chuỗi.

Các chế độ mở tập tin (*Open Flags*)

Khi mở một tệp bằng system call open(path, flags), chế độ mở được truyền dưới dạng các cờ bit kết hợp với nhau bằng toán tử |. Các cờ được định nghĩa trong kernel/fcntl.h bao gồm:

Cờ truy cập (*Flags*)	Ý nghĩa kỹ thuật và ngữ cảnh sử dụng trong xv6
O_RDONLY	Mở tập tin ở chế độ chỉ đọc (dùng khi đọc tệp nguồn trong cp, diff, duyệt thư mục trong tree, du).
O_WRONLY	Mở tập tin ở chế độ chỉ ghi (dùng khi ghi dữ liệu ra tệp đích trong cp).
O_RDWR	Mở tập tin cho phép đồng thời cả hai thao tác đọc và ghi dữ liệu.
O_CREATE	Tự động tạo tập tin mới nếu đường dẫn chưa tồn tại; nếu tệp đã tồn tại, giữ nguyên nội dung cũ.
O_TRUNC	Cắt cụt độ dài tập tin về 0 byte ngay khi mở, xóa toàn bộ nội dung cũ (thường kết hợp với O_WRONLY | O_CREATE trong cp).

Kỹ thuật duyệt cây thư mục đệ quy (Recursive Directory Traversal)

Các tiện ích như tree và du đòi hỏi khả năng duyệt toàn bộ cây phân cấp thư mục. Thuật toán duyệt đệ quy dựa trên mô hình duyệt đồ thị theo chiều sâu (DFS):

1. **Điều kiện dừng:** Khi đường dẫn hiện tại trỏ đến một tệp thông thường (st.type == T_FILE), chương trình xử lý thông tin tệp (cập nhật dung lượng hoặc in tên) rồi quay lui.
2. **Xử lý nút nhánh:** Khi đường dẫn trỏ đến một thư mục (st.type == T_DIR), chương trình mở thư mục và đọc tuần tự từng bản ghi struct dirent.
3. **Loại trừ liên kết tự thân và liên kết cha:** Điểm tối quan trọng trong việc duyệt cây thư mục là **bắt buộc phải bỏ qua hai mục đặc biệt là "." (thư mục hiện hành) và ".." (thư mục cha)**. Nếu không lọc hai mục này, hàm đệ quy sẽ lặp vô tận giữa thư mục cha và thư mục con cho đến khi làm tràn ngăn xếp bộ nhớ (*stack overflow*) hoặc chạm ngưỡng giới hạn mở tệp của hệ điều hành.
4. **Ghép đường dẫn an toàn:** Với mỗi mục con hợp lệ, chương trình tạo đường dẫn mới theo định dạng path/name. Cần bảo đảm bộ đệm chứa đường dẫn

đủ lớn (ít nhất 512 byte trong xv6) và kiểm tra giới hạn độ dài trước khi ghép chuỗi để ngăn chặn triệt để lỗi tràn bộ đệm (*buffer overflow*).

3.6 Xử lý dòng dữ liệu I/O và nguyên tắc quản trị tài nguyên an toàn

Để các chương trình vận hành ổn định trên các tập dữ liệu có quy mô lớn hoặc trong các tình huống ngoại lệ, nhóm đã áp dụng các nguyên tắc kỹ thuật chặt chẽ về quản trị I/O và vòng đời tài nguyên.

Xử lý I/O theo khối đệm (Chunk-based Buffer I/O)

Trong các tiện ích sao chép dữ liệu như cp, việc đọc hoặc ghi từng byte đơn lẻ thông qua system call sẽ làm gia tăng chi phí chuyển ngữ cảnh (trap overhead) lên mức không thể chấp nhận được. Ngược lại, việc đọc toàn bộ nội dung tệp vào một vùng nhớ lớn trong RAM lại bất khả thi đối với các tệp có dung lượng vượt quá kích thước bộ nhớ khả dụng của tiến trình.

Giải pháp chuẩn mực được áp dụng là **kỹ thuật truyền dữ liệu qua bộ đệm theo khối (*buffer streaming*)**. Chương trình duy trì một vùng đệm tĩnh có kích thước cố định (ví dụ: 512 byte, tương ứng đúng bằng kích thước một khối đĩa (*block size*) của hệ thống tệp xv6). Trong vòng lặp sao chép:

- Gọi n = read(src_fd, buf, sizeof(buf)). Giá trị $n > 0$ cho biết số byte thực tế đọc được; $n = 0$ thông báo đã chạm điểm kết thúc tệp (EOF); và $n < 0$ báo hiệu lỗi đọc từ đĩa.
- Khi $n > 0$, chương trình ghi đúng n byte vào tệp đích bằng write(dst_fd, buf, n). Cần kiểm tra kỹ giá trị trả về để bảo đảm toàn bộ dữ liệu đã được ghi trọn vẹn, phòng ngừa trường hợp đầy đĩa.

Kỹ thuật phân tích dòng văn bản trong diff

Tiện ích diff đòi hỏi phải so sánh hai tệp tin theo từng dòng văn bản độc lập. Do xv6 không có sẵn các hàm xử lý dòng mức cao như getline() trong POSIX, nhóm tự xây dựng cơ chế đệm dòng:

- Dữ liệu từ đĩa được đọc theo từng khối vào một bộ đệm vòng.
- Một hàm trợ giúp sẽ quét tìm ký tự xuống dòng ('\n'). Toàn bộ các ký tự từ vị trí hiện tại đến trước '\n' được gom thành một dòng dữ liệu hoàn chỉnh.
- Xử lý chặt chẽ các trường hợp biên (*edge cases*): dòng cuối cùng của tệp không kết thúc bằng ký tự '\n' trước khi gặp EOF; tệp rỗng hoàn toàn (0 byte); và trường hợp dòng văn bản có độ dài vượt quá kích thước bộ đệm

ban đầu (cần cấp phát động lại vùng nhớ bằng malloc() và di chuyển dữ liệu bằng memmove()).

Nguyên tắc giải phóng tài nguyên và xử lý lỗi

Môi trường thực thi của xv6 áp đặt các ràng buộc tài nguyên rất nghiêm ngặt: mỗi tiến trình chỉ được phép mở tối đa NOFILE = 16 tập tin mô tả đồng thời, và toàn bộ hệ thống chỉ có tối đa 64 tiến trình chạy cùng lúc. Do đó, việc quản trị tài nguyên phải tuân thủ tuyệt đối các nguyên tắc:

1. **Đóng tập tin mô tả ngay khi hoàn tất:** Mọi fd thu được từ open() hoặc pipe() đều phải được gọi close() tương ứng. Trong các hàm có nhiều nhánh thoát lỗi (*early exit*), việc dọn dẹp descriptor phải được thực hiện ở tất cả các nhánh.
2. **Đóng sớm các đầu pipe không dùng:** Trong các chương trình đa tiến trình (pingpong, primes), các đầu ghi và đọc không thuộc trách nhiệm của tiến trình hiện tại phải được đóng *ngay lập tức* sau lời gọi fork(), không giữ lại cho đến cuối chương trình.
3. **Thu hồi bộ nhớ động:** Mọi khối nhớ được cấp phát bằng malloc() phải có lệnh free() đối ứng sau khi dùng xong để ngăn ngừa rò rỉ bộ nhớ (*memory leak*).
4. **Thông báo lỗi qua stderr và mã thoát chuẩn:** Khi một thao tác gặp sự cố (sai cú pháp tham số, không mở được tệp, lỗi đọc/ghi), chương trình luôn in thông báo chi tiết ra tập tin mô tả số 2 (stderr) thông qua fprintf(2, ...) và thoát với mã trạng thái khác 0 (exit(1)), cho phép các tiến trình gọi hoặc shell nhận biết chính xác trạng thái thực thi.

4. Các chương trình đã thực hiện

4.1 sleep

Mục tiêu.

- **Chức năng:** Tạm dừng tiến trình hiện tại trong số chu kỳ đồng hồ do ticks chỉ định.
- **Tham số:** Chấp nhận đúng một tham số bắt buộc theo cú pháp sleep ticks; ticks phải là một giá trị số hợp lệ.
- **Xử lý lỗi:** Phát hiện trường hợp thiếu, thừa hoặc sai định dạng tham số và thông báo lỗi phù hợp.

Kiến thức sử dụng.

- **Tick và thời gian hệ thống:** Tick là đơn vị đếm theo nhịp ngắt của bộ định thời trong xv6. Số tick cho biết khoảng cần chờ, nhưng không thể hiểu trực tiếp là số giây.
- **Trạng thái chờ của tiến trình:** Một tiến trình có thể tạm ngừng thực thi cho đến khi điều kiện chờ được đáp ứng. System call sleep() sử dụng cơ chế này, để tiến trình nhường CPU thay vì liên tục kiểm tra đồng hồ.
- **Chuyển đổi tham số số:** Tham số dòng lệnh ban đầu là chuỗi ký tự, kể cả khi người dùng nhập một số. Kiểm tra chuỗi trước khi chuyển đổi giúp phân biệt giá trị số với dữ liệu sai định dạng.

Ý tưởng triển khai. Lệnh sleep sử dụng cơ chế chờ của hệ điều hành thay vì tự đếm thời gian bằng một vòng lặp. Sau khi kiểm tra tham số không rỗng và chỉ chứa chữ số, chương trình chuyển giá trị nhập thành số tick rồi yêu cầu tạm dừng tiến trình. Trong thời gian đó, tiến trình không chiếm CPU để chờ, nên hệ thống có thể chạy các công việc khác. Hết khoảng chờ, tiến trình được phép chạy tiếp và kết thúc. Thời lượng ở đây được tính theo tick của xv6, không phải theo giây.

Mã giả dưới đây mô tả các bước kiểm tra tham số và tạm dừng tiến trình trong hàm main().

sleep

```
1  PROCEDURE Main(arguments)
2      IF arguments do not contain exactly one value THEN
3          REPORT usage error; EXIT 1
4      END IF
5      value ← the supplied argument
6      IF value is empty or contains a non-digit character THEN
7          REPORT invalid duration; EXIT 1
8      END IF
9      ticks ← ConvertToInteger(value)
10     Suspend the current process for ticks clock ticks
11     EXIT 0
12 END PROCEDURE
```

4.2 pingpong

Mục tiêu.

- **Chức năng:** Minh họa giao tiếp hai chiều giữa tiến trình cha và tiến trình con bằng cách gửi “ping” và nhận phản hồi “pong”.

- **Tham số:** Không nhận tham số dòng lệnh; lệnh được thực thi trực tiếp bằng pingpong.
- **Kết quả:** In thông báo giao tiếp kèm PID của các tiến trình và kết thúc bình thường sau khi trao đổi hoàn tất.

Kiến thức sử dụng.

- **Tiến trình cha và con:** Sau fork(), cha và con tiếp tục chạy độc lập từ cùng vị trí nhưng nhận giá trị trả về khác nhau. Nhờ đó, chương trình phân chia được vai trò gửi và phản hồi cho hai tiến trình.
- **Pipe một chiều:** Một pipe chỉ truyền dữ liệu theo một hướng, từ đầu ghi đến đầu đọc. Lượt trao đổi cần cả gửi lẫn phản hồi nên sử dụng hai pipe, mỗi pipe đảm nhiệm một chiều.
- **Đồng bộ tiến trình:** Thao tác đọc pipe có thể chờ khi chưa có dữ liệu, còn wait() cho phép cha chờ con kết thúc. Hai cơ chế này xác lập quan hệ trước–sau giữa các công việc mà không phụ thuộc tiến trình nào được cấp CPU trước.
- **Các đầu pipe được kế thừa:** Tiến trình con thừa hưởng các đầu pipe đã mở trước khi được tạo. Mỗi bên cần đóng những đầu không dùng và chỉ giữ đầu đọc, đầu ghi phù hợp với vai trò của mình.

Ý tưởng triển khai. Hai tiến trình trao đổi qua hai pipe, một pipe cho chiều cha gửi sang con và một pipe cho chiều phản hồi. Tiến trình cha gửi một byte để bắt đầu lượt trao đổi. Khi nhận được byte này, tiến trình con in thông báo nhận ping, gửi lại byte đó rồi kết thúc. Cha chờ con hoàn tất, đọc phản hồi và in thông báo nhận pong. Nhờ trình tự gửi–nhận này, thông báo ping luôn xuất hiện trước pong mà không cần quy định tiến trình nào phải được chạy trước. Các đầu pipe không sử dụng được đóng ở mỗi bên; những đầu còn lại cũng được đóng sau khi trao đổi xong.

Trình tự trao đổi giữa tiến trình cha và con được trình bày trong mã giả dưới đây, tương ứng với hàm main().

pingpong

```
1  PROCEDURE Main()
2      Create channels p (parent to child) and q (child to parent)
3      Create a child process
4      IF channel creation or process creation fails THEN
5          REPORT error; EXIT 1
6      END IF
7      IF executing in the parent THEN
8          Close unused channel ends
```

```
    Send one byte through p; close the sending end
    Wait for the child to finish
    Receive one byte through q
    OUTPUT parent identifier and "received pong"
    Close the receiving end
  ELSE
    Close unused channel ends
    Receive one byte through p
    OUTPUT child identifier and "received ping"
    Send the same byte through q
    Close the remaining channel ends
  END IF
  On any send or receive failure, report error and exit 1
  EXIT 0
END PROCEDURE
```

4.3 primes

Mục tiêu.

- **Chức năng:** Sinh và in các số nguyên tố trong phạm vi từ 2 đến 280.
- **Tham số:** Không nhận tham số dòng lệnh; phạm vi xử lý được cố định theo yêu cầu của bài thực hành.
- **Mục tiêu xử lý:** Thể hiện mô hình sàng Eratosthenes đồng thời và bảo đảm mỗi số nguyên tố được xuất đúng một lần.

Kiến thức sử dụng.

- **Sàng số nguyên tố:** Mọi hợp số đều có ít nhất một ước nguyên tố nhỏ hơn nó. Loại dần các bội của những số nguyên tố đã tìm được sẽ giữ lại các ứng viên cho bước sàng tiếp theo.
- **Pipeline tiến trình:** Pipeline chia công việc thành các bước nối tiếp, trong đó đầu ra của bước trước là đầu vào của bước sau. Với bài này, mỗi tiến trình đảm nhiệm một bộ lọc và truyền số qua pipe, không cần dùng chung một mảng dữ liệu.
- **EOF và kết thúc chuỗi xử lý:** Bên đọc nhận EOF khi dữ liệu trong pipe đã hết và mọi đầu ghi đã đóng; vì vậy, mỗi bước chỉ giữ những đầu pipe cần dùng. Việc chờ tiến trình con hoàn tất giúp thu hồi thông tin tiến trình đã kết thúc và bảo đảm phần việc phía sau hoàn thành trước khi bước hiện tại thoát.

Ý tưởng triển khai. Các số từ 2 đến 280 được đưa lần lượt qua một chuỗi tiến

trình, mỗi tiến trình thực hiện một bước sàng. Bước đầu lấy 2 làm số nguyên tố và loại các số chia hết cho 2. Những số còn lại được chuyển sang bước sau, nơi 3 được chọn và các bội của 3 tiếp tục bị loại. Cách xử lý này lặp lại cho đến khi không còn số cần xét. Số đầu tiên ở mỗi bước là số nguyên tố: nếu nó chia hết cho một số nguyên tố nhỏ hơn, nó đã bị loại ở bước sàng trước đó.

Các bước sàng nối với nhau bằng pipe và xử lý từng số khi nhận được, không cần gom toàn bộ dãy còn lại vào bộ nhớ. Tiến trình kế tiếp chỉ được tạo khi có số vượt qua bước lọc hiện tại. Sau khi gửi hết dữ liệu, tiến trình đóng đầu ghi để báo hết dữ liệu cho bước sau, rồi chờ bước đó hoàn tất trước khi thoát.

Mã giả dưới đây mô tả một bước sàng trong hàm primes(left). Các bước tiếp theo áp dụng cùng cách xử lý cho những số còn lại.

primes

```
PROCEDURE Primes(left)
    prime ← ReceiveInteger(left)
    IF a complete integer was not received THEN
        Close left; EXIT 0
    END IF
    OUTPUT "prime" followed by prime
    right ← no outgoing channel
    WHILE a complete integer n can be received from left DO
        IF n is divisible by prime THEN
            CONTINUE
        END IF
        IF right has not been created THEN
            Create a channel and a child process
            IF creation fails THEN
                REPORT error; EXIT 1
            END IF
            Child closes unused ends and runs Primes(new channel)
            Parent keeps the sending end as right
        END IF
        Send n through right
        IF sending fails THEN
            REPORT error; EXIT 1
        END IF
    END WHILE
    Close left
    IF right exists THEN
        Close right; wait for the child to finish
    END IF
    EXIT 0
END PROCEDURE
```

4.4 cp

Mục tiêu.

- **Chức năng:** Sao chép toàn bộ nội dung từ tệp nguồn src sang tệp đích dst.
- **Tham số:** Nhận đúng hai tham số đường dẫn theo cú pháp cp src dst.
- **Xử lý lỗi:** Kiểm tra nguồn không hợp lệ, đích không thể tạo hoặc mở, trường hợp nguồn và đích cùng trỏ tới một tệp, đồng thời báo lỗi khi cú pháp không đúng.

Kiến thức sử dụng.

- **Bộ mô tả tệp:** Khi mở một tệp, tiến trình nhận một số nguyên dùng để truy cập tệp đó trong các thao tác đọc, ghi và đóng. Hai tệp nguồn và đích được mở với chế độ khác nhau, phù hợp với chiều sao chép.
- **Đọc–ghi theo khối:** Bộ đệm giữ một phần dữ liệu trong lúc chuyển từ nguồn sang đích, nên không cần nạp toàn bộ tệp vào bộ nhớ. Số byte thực sự đọc được quyết định lượng cần ghi, đặc biệt ở phần cuối tệp.
- **Nhận diện tệp và liên kết cứng:** Hai đường dẫn khác nhau có thể trỏ đến cùng một tệp thông qua liên kết cứng. Cặp thông tin thiết bị và số inode, tức mã nhận diện tệp trên thiết bị, cho phép phát hiện trường hợp này trước khi ghi đè nội dung.

Ý tưởng triển khai. Trước khi sao chép, cần xác định nguồn và đích có phải là hai tệp khác nhau hay không. Chỉ so sánh đường dẫn là chưa đủ, bởi hai tên có thể cùng trỏ đến một tệp. Chương trình vì vậy kiểm tra thông tin nhận diện của tệp và loại tệp trước khi xóa nội dung đích. Nguồn phải là tệp thông thường; nếu đích đã tồn tại thì đích cũng phải đáp ứng điều kiện này.

Dữ liệu sau đó được đọc từ nguồn và ghi sang đích theo từng khối tối đa 512 byte. Mỗi lượt ghi đúng lượng dữ liệu vừa đọc được, nên phần cuối của tệp vẫn được sao chép đầy đủ dù ngắn hơn một khối. Việc sao chép tuần tự giữ nguyên thứ tự các byte và không đòi hỏi bộ nhớ tăng theo kích thước tệp. Khi gặp lỗi đọc hoặc ghi, chương trình dừng và báo lỗi; dữ liệu đã ghi vào đích trước đó không được khôi phục.

Quy trình kiểm tra tệp và sao chép dữ liệu trong hàm main() được tóm tắt bằng mã giả dưới đây.

cp

```
PROCEDURE Main(arguments)
    Require exactly two arguments: source and destination
    Open source and verify that it is a regular file
    IF destination already exists THEN
        Verify that it is a different regular file
    END IF
    IF an argument or file check fails THEN
        Close opened files; report error; EXIT 1
    END IF
    Create or open destination, replacing its old contents
    IF destination cannot be opened THEN
        Close source; report error; EXIT 1
    END IF
    WHILE reading source yields a nonempty portion DO
        Write exactly that portion to destination
        IF the full portion was not written THEN
            Close both files; report error; EXIT 1
        END IF
    END WHILE
    IF the last read failed THEN
        Close both files; report error; EXIT 1
    END IF
    Close both files
    EXIT 0
END PROCEDURE
```

4.5 tree

Mục tiêu.

- **Chức năng:** Hiển thị thư mục và tệp tin theo dạng cây phân cấp.
- **Tham số:** Nhận đường dẫn tùy chọn path, mặc định là thư mục hiện hành .; hỗ trợ -L depth để giới hạn độ sâu và -d để chỉ liệt kê thư mục.
- **Xử lý lỗi:** Phát hiện và thông báo rõ ràng các đường dẫn, tham số hoặc tùy chọn không hợp lệ.

Kiến thức sử dụng.

- **Cây thư mục:** Một thư mục chứa các mục trỏ đến tệp hoặc thư mục con, tạo nên cấu trúc phân cấp. Biết loại của từng mục giúp xác định nhánh nào có thể tiếp tục duyệt; trong xv6, thông tin theo đường dẫn được lấy qua hàm thư viện stat().
- **Đệ quy và duyệt chiều sâu:** Mỗi thư mục con có thể được xử lý theo cùng

cách với thư mục chứa nó. Đệ quy cho phép đi sâu vào một nhánh rồi quay lại các mục còn lại, đồng thời dùng độ sâu hiện tại để xác định mức thụt lề.

- **Điều kiện dừng khi duyệt:** Hai mục . và .. trỏ đến thư mục hiện tại và thư mục cha, nên không được xem là nhánh mới. Giới hạn độ sâu là một điều kiện dừng khác, cho phép chỉ duyệt đến số tầng người dùng yêu cầu.

Ý tưởng triển khai. Từ thư mục được chọn, chương trình duyệt theo chiều sâu: in tên một mục, duyệt hết nội dung bên trong nếu đó là thư mục, rồi mới chuyển sang mục kế tiếp. Mức thụt lề tăng theo độ sâu, thể hiện vị trí của từng mục trong cây. Hai mục . và .. được bỏ qua để không quay lại thư mục hiện tại hoặc thư mục cha; các mục chưa sử dụng cũng không được đưa vào kết quả.

Tùy chọn -L giới hạn số tầng được hiển thị, còn -d chỉ giữ lại các thư mục. Khi đạt độ sâu cho phép, chương trình không duyệt sâu hơn nhánh đó. Nếu đường dẫn đầu vào là một tệp thay vì thư mục, kết quả chỉ gồm tên tệp, trừ trường hợp đang bật -d.

Mã giả dưới đây thể hiện cách duyệt và hiển thị một nhánh thư mục, dựa trên hàm tree_traverse(path, level).

tree

```
1  PROCEDURE TreeTraverse(path, level)
2    IF a depth limit is set and level reaches it THEN
3      RETURN
4    END IF
5    Open directory path
6    IF opening fails THEN
7      REPORT error; RETURN
8    END IF
9    IF child paths would exceed the allowed length THEN
10     Close directory; report error; RETURN
11   END IF
12   FOR EACH entry read from directory DO
13     IF entry is unused, "." or ".." THEN
14       CONTINUE
15     END IF
16     childPath ← Join(path, entry.name)
17     IF information for childPath cannot be obtained THEN
18       REPORT error; CONTINUE
19     END IF
20     IF directories-only mode excludes entry THEN
21       CONTINUE
22     END IF
23     Display entry.name with indentation for level + 1
24     IF entry is a directory THEN
25       TreeTraverse(childPath, level + 1)
```

```
    END IF
  END FOR
  Close directory
END PROCEDURE
```

4.6 du

Mục tiêu.

- **Chức năng:** Tính và hiển thị dung lượng sử dụng của tệp hoặc thư mục theo đơn vị byte.
- **Tham số:** Nhận đường dẫn tùy chọn path, mặc định là .; hỗ trợ -s để chỉ in tổng dung lượng và -a để hiển thị thêm dung lượng của từng tệp.
- **Đầu ra:** Hiển thị dung lượng theo byte cùng đường dẫn tương ứng và tuân theo chế độ hiển thị đã chọn.

Kiến thức sử dụng.

- **Thông tin kích thước tệp:** Hệ thống tệp lưu kích thước của từng tệp trong thông tin quản lý của nó. Lấy giá trị này đủ để thống kê số byte, không cần đọc lại toàn bộ nội dung tệp.
- **Duyệt hậu thứ tự:** Khi kết quả của thư mục cha phụ thuộc các mục con, các mục con phải được tính trước. Cách duyệt này cho phép mỗi nhánh trả về một tổng để cộng vào thư mục chứa nó.
- **Kích thước dữ liệu và dung lượng đĩa:** Số byte của tệp không đồng nhất với số khối đĩa được cấp phát để lưu tệp. Phép tính trong bài cộng kích thước các tệp theo byte, không cộng kích thước bản thân thư mục hay quy đổi sang dung lượng khối đĩa.

Ý tưởng triển khai. Dung lượng của một thư mục được tính từ các tệp và thư mục con bên trong. Với tệp thông thường hoặc tệp thiết bị, chương trình lấy kích thước do hệ thống ghi nhận. Với thư mục con, chương trình tính xong tổng của nhánh đó rồi cộng vào thư mục cha. Kết quả vì vậy được tổng hợp từ dưới lên: các mục con hoàn tất trước, thư mục chứa chúng hoàn tất sau.

Các tùy chọn chỉ thay đổi phần kết quả được in, không thay đổi cách cộng dung lượng. Chế độ mặc định in tổng của từng thư mục; -a in thêm từng tệp, còn -s chỉ in tổng của đường dẫn được chọn. Trong bài này, dung lượng là tổng kích thước theo byte, không tính kích thước bản thân thư mục hay số khối đĩa thực tế sử dụng. Nhánh không truy cập được sẽ được báo lỗi và bỏ qua khi tính tổng.

Cách tính dung lượng và cộng dồn kết quả từ các mục con được trình bày trong mã giả của hàm du(path) dưới đây.

du

```
PROCEDURE Du(path)
    Obtain information about path
    IF information cannot be obtained THEN
        REPORT error; RETURN 0
    END IF
    IF path is a regular file or a device THEN
        IF all-files mode is enabled and summary mode is off THEN
            OUTPUT size in bytes and path
        END IF
        RETURN size in bytes
    END IF
    IF path is not a directory THEN
        RETURN 0
    END IF
    Open directory and check the allowed child-path length
    IF opening or the length check fails THEN
        Close directory if opened; report error; RETURN 0
    END IF
    total ← 0
    FOR EACH entry read from directory DO
        IF entry is unused, "." or ".." THEN
            CONTINUE
        END IF
        total ← total + Du(Join(path, entry.name))
    END FOR
    Close directory
    IF summary mode is off THEN
        OUTPUT total and path
    END IF
    RETURN total
END PROCEDURE
```

4.7 diff

Mục tiêu.

- **Chức năng:** So sánh nội dung của hai tệp theo từng dòng và xác định các vị trí khác biệt.
- **Tham số:** Nhận hai tệp bắt buộc file1, file2 và tùy chọn -q theo cú pháp diff file1 file2 [-q].
- **Đầu ra:** Ở chế độ thông thường, chỉ ra các dòng khác biệt của từng tệp; với

-q, chỉ thông báo hai tệp có khác nhau hay không.

Kiến thức sử dụng.

- **So sánh theo vị trí:** Hai dòng được đối chiếu khi có cùng số thứ tự trong hai tệp. Đây là phép so sánh từng cặp, không tìm cách ghép lại các dòng tương ứng khi có chèn hoặc xóa nội dung.
- **Dòng trống, EOF và dấu xuống dòng:** Dòng trống vẫn là một dòng, còn EOF cho biết không còn dữ liệu để đọc. Dấu xuống dòng cũng là một phần cần kiểm tra, vì dòng cuối có thể kết thúc ngay tại EOF mà không có dấu này.
- **Bộ đệm động:** Độ dài dòng không biết trước, nên vùng nhớ chứa dòng cần được mở rộng khi chưa đủ chỗ. Theo dõi số byte thực tế giúp giữ đủ nội dung, kể cả khi có byte 0 nằm giữa dòng; vùng nhớ được giải phóng sau khi sử dụng.

Ý tưởng triển khai. Hai tệp được đọc song song và so sánh các dòng có cùng số thứ tự. Ngoài nội dung, chương trình kiểm tra độ dài và dấu xuống dòng, đồng thời phân biệt dòng trống với trường hợp tệp đã hết dữ liệu. Do đó, hai dòng có cùng phần chữ nhưng khác dấu xuống dòng cuối tệp vẫn được xem là khác nhau. Bộ đệm được mở rộng theo độ dài dòng để không cắt mất phần nội dung cần so sánh.

Với mỗi khác biệt, kết quả ghi tên tệp, số dòng và nội dung của hai phía. Nếu một tệp kết thúc trước, các dòng còn lại ở tệp kia được đối chiếu với trạng thái hết tệp. Khi có -q, chương trình dừng ở khác biệt đầu tiên và chỉ in thông báo. Phép so sánh trong bài không căn chỉnh lại dòng khi có chèn hoặc xóa: một dòng thêm vào giữa tệp có thể khiến nhiều cặp dòng phía sau được báo khác nhau.

Mã giả dưới đây mô tả quy trình so sánh trong hàm main(), từ việc đọc từng cặp dòng đến ghi nhận và hiển thị khác biệt.

diff

```
1  PROCEDURE Main(arguments)
2      Parse two paths and optional quiet flag at either end
3      Reject invalid syntax or paths beginning with a hyphen
4      Open both files and verify that they are regular files
5      On a setup failure, close opened files and exit 2
6      status ← 0; number ← 1
7      LOOP
8          first ← ReadLine(firstFile)
9          second ← ReadLine(secondFile)
10         IF either line could not be read THEN
```

```
11          REPORT error; status ← 2; BREAK
12        END IF
13        IF neither file has another line THEN
14          BREAK
15        END IF
16        IF presence, length, newline or contents differ THEN
17          status ← 1
18          IF quiet mode is enabled THEN
19            OUTPUT "diff: files differ"; BREAK
20          END IF
21          PrintLine(firstFile, number, first)
22          PrintLine(secondFile, number, second)
23          IF either output operation fails THEN
24            REPORT error; status ← 2; BREAK
25          END IF
26        END IF
27        number ← number + 1
28      END LOOP
29      Release stored lines and close both files
30      EXIT status
31    END PROCEDURE
```

5. Kết quả kiểm thử tổng hợp

5.1 Môi trường và cách thực hiện

Kiểm thử được thực hiện tự động trên xv6 chạy bằng QEMU 8.2.2, với 128 MB bộ nhớ và 3 CPU mô phỏng. Mã nguồn được biên dịch trên Ubuntu 24.04 trong WSL bằng GCC 13.3.0 cho RISC-V. Bộ chạy kiểm thử tạo một bản sao tạm của xv6, biên dịch lại chương trình và tạo ảnh hệ thống tệp riêng; vì vậy, các tệp thử nghiệm không làm thay đổi fs.img trong thư mục làm việc.

Bộ kiểm thử gồm cptest, difftest, linetest và coretest, đặt tại tests/lab01/. Các bài kiểm tra đối chiếu kết quả với giá trị mong đợi, bao gồm nội dung tệp, thông báo xuất ra và mã thoát. Riêng sleep, thời gian chờ được đo bằng bộ đếm tick của xv6. Lệnh chạy từ thư mục gốc của dự án, trong Ubuntu WSL, là:

```
python3 tests/lab01/run.py \
  --log docs/test-results/lab01-qemu.log
```

5.2 Kết quả theo từng chương trình

Lần chạy ghi nhận **77/77 phép kiểm tra đạt**, không xuất hiện thông báo FAIL và bộ chạy kiểm thử kết thúc với mã 0. Con số này bao gồm các lần chạy lặp lại và một kiểm tra tạo liên kết cứng phục vụ thử nghiệm cp; không phải số lượng trường hợp độc lập hay tỷ lệ bao phủ mã nguồn.

Đối tượng	Phạm vi kiểm tra	Kết quả ghi nhận	Đạt/tổng
sleep	Chờ 5 tick, 0 tick; thiếu, thừa, sai định dạng và số âm	Lượt sleep 5 đo được 5 tick; thông báo và mã thoát đúng kỳ vọng	6/6
pingpong	Chạy 3 lần; kiểm tra thứ tự thông báo và PID	Mỗi lần in ping trước pong; hai PID khác nhau; mã thoát 0	3/3
primes	Chạy 2 lần; đối chiếu dãy bằng phép thử chia độc lập	Đủ 59 số nguyên tố từ 2 đến 280, đúng thứ tự, không thừa hoặc trùng	2/2
cp	Tệp 0–4097 byte; ghi đè; cùng tệp, liên kết cứng; đối số và đường dẫn lỗi	Đúng từng byte và kích thước; giữ nguyên nguồn trong các ca từ chối sao chép	18/18
tree	Cây nhiều tầng; -d, -L 1, -L 2; tệp, thư mục rỗng và đầu vào lỗi	Cấu trúc, thụt lề, giới hạn độ sâu, thông báo và mã thoát đúng kỳ vọng	11/11
du	Tổng thư mục; -a, -s, kết hợp hai cờ; tệp đơn và đầu vào lỗi	Tệp mẫu 3, 5, 2 byte cho tổng 10 byte; kết quả chi tiết và tóm tắt đúng	8/8
diff	Tệp rỗng; dòng khác nhau; -q; hết tệp, dấu xuống dòng; dòng dài và byte 0	Nội dung xuất và mã thoát 0/1/2 đúng kỳ vọng, kể cả dòng 1200 byte	20/20
Đọc dòng	Dòng trống, dòng cuối, hết tệp, mở rộng bộ đệm, byte 0 và lỗi đọc	Phân biệt dòng trống với hết tệp; giữ đủ nội dung và nhận biết lỗi đọc	9/9
Tổng		**Tất cả phép kiểm tra đều đạt**	**77/77**

5.3 Nhật ký thực thi và phạm vi kết luận

Nhật ký QEMU đầy đủ được lưu tại docs/test-results/lab01-qemu.log. Trích các dòng xác nhận hoàn tất từ lần chạy thực tế:

```
ALL CP TESTS PASSED
ALL DIFF TESTS PASSED
ALL LINE TESTS PASSED
MEASURE: sleep 5 elapsed=5 ticks
ALL SLEEP TESTS PASSED
ALL PINGPONG TESTS PASSED
ALL PRIMES TESTS PASSED
ALL TREE TESTS PASSED
ALL DU TESTS PASSED
ALL CORE TESTS PASSED
```

Các kết quả trên xác nhận hành vi của chương trình trong những trường hợp đã kiểm tra, không chứng minh chương trình không còn lỗi. Bộ test chưa mô phỏng đầy đĩa, thiếu bộ nhớ, cạn tài nguyên tiến trình hoặc mọi giới hạn đường dẫn; mẫu sao chép lớn nhất là 4097 byte và mẫu dòng dài là 1200 byte. Thời gian 5 tick là số đo của lượt chạy này, không phải cam kết tiến trình luôn được lập lịch lại đúng thời điểm trên mọi lần chạy.

Đây là **bộ kiểm thử riêng cho phạm vi Lab 01 của báo cáo**, không phải kết quả make grade. Bộ chấm grade-lab-util có trong kho còn kiểm tra find và xargs, nhưng không bao gồm cp, tree, du, diff. Vì chưa chạy toàn bộ bộ chấm đó trong lần kiểm thử này, báo cáo không ghi điểm chấm MIT hoặc quy đổi 77/77 thành điểm trên 100.

6. Khó khăn gặp phải và giải pháp khắc phục

1. **Vấn đề 1: Tràn số lượng File Descriptor trong bài primes.**

   Nguyên nhân: Tiến trình con kế thừa toàn bộ bảng file descriptor của tiến trình cha nhưng không đóng kịp thời các đầu ghi của các stage trước đó, dẫn đến cạn kiệt tài nguyên (vượt quá NOFILE trong kernel).

   Giải pháp: Đóng ngay lập tức các đầu pipe không dùng ở cả cha và con trước khi bước vào vòng lặp xử lý dữ liệu.

2. **Vấn đề 2: Lặp vô hạn khi duyệt thư mục trong tree và du.**

   Nguyên nhân: Bỏ sót việc lọc hai liên kết đặc biệt là thư mục hiện hành (.)

và thư mục cha (..).

Giải pháp: Bổ sung kiểm tra strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0 trước khi gọi hàm đệ quy.

7. Kết luận và hướng phát triển

Qua bài thực hành Lab 01, nhóm đã hoàn thiện toàn diện các kỹ năng lập trình không gian người dùng trên hệ điều hành xv6:

- Nắm vững cơ chế vận hành của system call và sự chuyển giao giữa User Space và Kernel Space.
- Làm chủ kỹ thuật điều khiển luồng tiến trình và đồng bộ hóa thông qua các cơ chế IPC cơ bản nhưng hiệu quả.
- Hiểu rõ cấu trúc tổ chức và lưu trữ dữ liệu của hệ thống tệp tin xv6 ở mức độ nhị phân.

Tài liệu tham khảo

[1] Russ Cox, Frans Kaashoek, Robert Morris, *xv6: a simple, Unix-like teaching operating system*, bản RISC-V, ngày 31/08/2024. Tham khảo chương 1 về system call và pipe, chương 8 về hệ thống tệp. https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf.

[2] MIT, học phần 6.1810, *Lab: Xv6 and Unix utilities*, Fall 2024. Yêu cầu và hướng dẫn cho sleep, pingpong, primes. https://pdos.csail.mit.edu/6.1810/2024/labs/util.html.

[3] Tài liệu hướng dẫn thực hành Hệ điều hành Lab 01 do Giảng viên bộ môn biên soạn.

[4] MIT PDOS, *xv6-riscv*, kho mã nguồn chính thức. Tham khảo cách tổ chức tiến trình, pipe, hệ thống tệp và chương trình ls. Mã nguồn thực hiện bài được đối chiếu trực tiếp trong thư mục xv6-labs-2024/ của dự án. https://github.com/mit-pdos/xv6-riscv.

[5] Russ Cox, *Bell Labs and CSP Threads*. Tham khảo mô hình sàng số nguyên tố bằng các tiến trình nối với nhau qua pipe. https://swtch.com/~rsc/thread/.

[6] MIT, học phần 6.1810, *Tools*, Fall 2024. Hướng dẫn chuẩn bị công cụ RISC-V, QEMU và môi trường chạy xv6.
https://pdos.csail.mit.edu/6.1810/2024/tools.html.

[7] GNU Project, *GNU Coreutils Manual*, các mục *cp invocation* và *du invocation*. Tham khảo chức năng và cách sử dụng các tiện ích sao chép, thống kê dung lượng trên Unix.
https://www.gnu.org/software/coreutils/manual/coreutils.html.

[8] GNU Project, *Comparing and Merging Files*, mục *Invoking diff*. Tham khảo cách diễn giải kết quả so sánh và mã thoát của diff.
https://www.gnu.org/software/diffutils/manual/html_node/Invoking-diff.html.

[9] Microsoft Learn, *How to install Linux on Windows with WSL*. Tham khảo yêu cầu hệ thống, cài bản phân phối Ubuntu và kiểm tra phiên bản WSL.
https://learn.microsoft.com/en-us/windows/wsl/install.

[10] Homebrew, *Installation*. Tham khảo cách cài đặt và yêu cầu môi trường macOS.
https://docs.brew.sh/Installation.

[11] Homebrew Formulae, *riscv64-elf-gcc* và *qemu*. Tham khảo tên gói, phụ thuộc Binutils và công cụ biên dịch RISC-V trên macOS.
https://formulae.brew.sh/formula/riscv64-elf-gcc;
https://formulae.brew.sh/formula/qemu.

Các tài liệu GNU được dùng để tham khảo hành vi của tiện ích Unix; các chương trình trong bài chỉ thực hiện phạm vi yêu cầu của Lab 01, không có đầy đủ tính năng của phiên bản GNU.